

63. Une table gravée

Ce qui ne change jamais n'a rien à faire en mémoire.

1. Ce qu'il faut comprendre

`const uint8_t T[]` est **gravé dans la cartouche**, comme une tuile. Le compilateur l'annonce : ces octets sortent du décompte de la mémoire de travail.

Ôter le `const` la met en mémoire — et la rend modifiable, et la rend vide au démarrage. Les deux formes ont leur usage ; la différence se voit dans le compte rendu de compilation, et nulle part ailleurs.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **242 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage – la mise en route de la console
main()         58 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
AdresseCase    10 o ← poser() – trouve l'adresse d'une case de l'écran
CopierTuiles   31 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles         16 o ← CopierTuiles – 1 lettre sur 43
COLONNES_0     6 o ← poser() – une table gravée
LIGNES_1       6 o ← poser() – une table gravée
```

Lettres gravées : A

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
poser() × 1    33 o sur place, dans main() + AdresseCase (10 o), COLONNES_0 (6 o), LIGNES_1 (6 o)
image() × 1    3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : pose 6 tuiles à des places écrites dans deux tableaux.
//
// const devant un tableau : il est GRAVÉ dans la cartouche et ne change jamais.
// Les deux tableaux vont ensemble : la place numéro i est (COLONNES[i], LIGNES[i]).

#include <poser> // pose une tuile sur une case du fond
```

2. UNE TABLE GRAVÉE DANS LA CARTOUCHE — COLONNES

```
const uint8_t COLONNES[] = { 2, 5, 8, 11, 14, 17 }; // les colonnes des 6 places
```

3. UNE TABLE GRAVÉE DANS LA CARTOUCHE — LIGNES

```
const uint8_t LIGNES[] = { 3, 6, 9, 6, 3, 6 }; // les lignes des 6 places
```

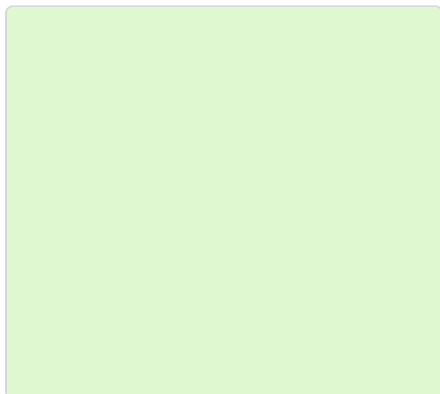
4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() { // Le jeu commence ici.
    for (uint8_t i = 0; i < sizeof(COLONNES); i++) // sizeof = le nombre de cases : 6.
        poser(COLONNES[i], LIGNES[i], 1); // Une tuile 1 (le A) à la place i.
                                           // i = 0 → (2, 3) ; i = 2 → (8, 9) ...

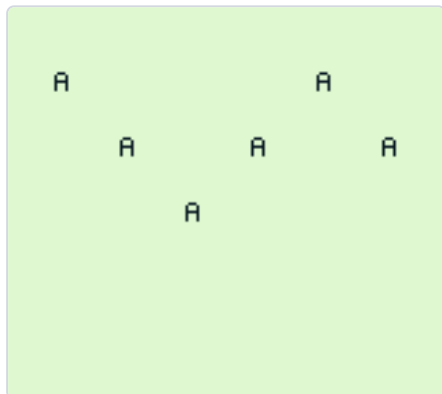
    while (true) { // Répète pour toujours :
        image(); // attend l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Six tuiles posées en zigzag, aux positions de la table.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la première est posée
- ✓ la quatrième aussi
- ✓ la dernière ferme le zigzag
- ✓ et rien n'est posé entre deux